

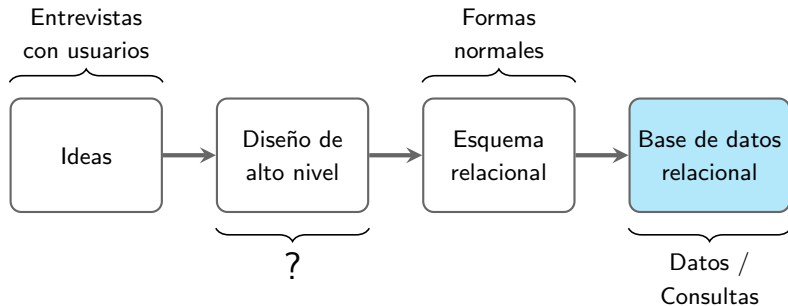
Modelo entidad/relación

Clase 11

IIC2413 - Sección 3

Prof. Cristian Riveros

Proceso de creación de una base de datos



El **modelo entidad/relación** permite hacer un diseño de alto nivel y traspasar las ideas de los usuarios a un esquema relacional

Outline

Modelo E/R

De modelo a esquema

Como llegar a un buen diseño

Llaves foráneas

Outline

Modelo E/R

De modelo a esquema

Como llegar a un buen diseño

Llaves foráneas

Diseño conceptual de una base de datos

Buscamos responder

- ¿Qué entidades modelar?
- ¿Cómo se relacionan estas entidades?
- ¿Cuáles son las restricciones del dominio?
- ¿Cómo lograr un buen diseño?

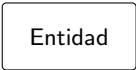
Para responder estas preguntas ocuparemos el **modelo entidad/relación**

Modelo entidad/relación

Definición

El **modelo E/R** esta basado en tres elementos principales:

1. Entidades



Entidad

2. Atributos



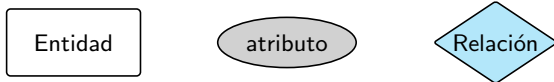
atributo

3. Relaciones

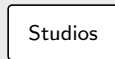
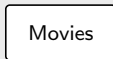


Relación

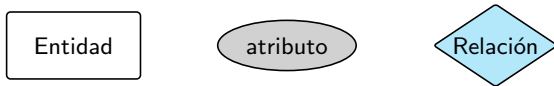
Modelo entidad/relación



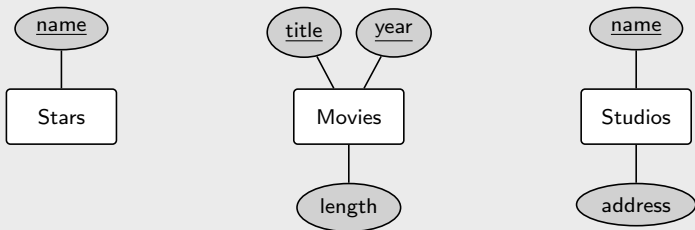
Ejemplo de entidades



Modelo entidad/relación

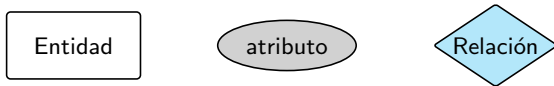


Ejemplo de entidades y atributos

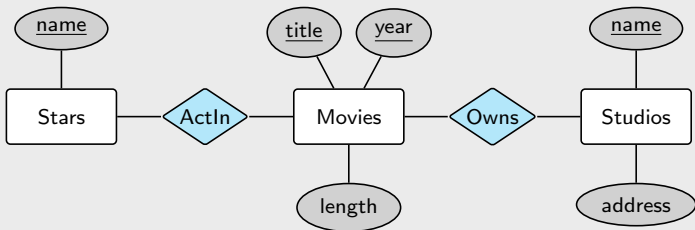


Usamos subrayado para marcar los atributos que son parte de la **llave**

Modelo entidad/relación

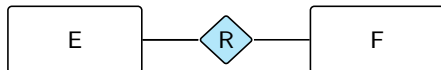


Ejemplo de entidades, atributos y relaciones

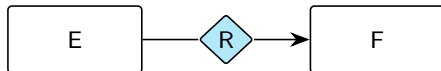


Modelo E/R: tipos de relaciones

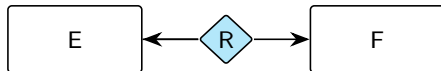
Relación varios-a-varios



Relación varios-a-uno

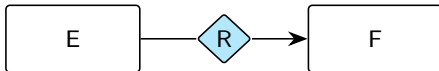


Relación uno-a-uno

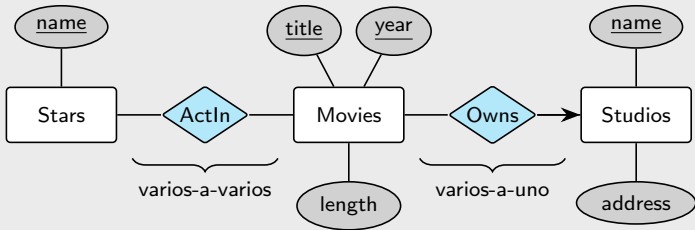


Modelo E/R: tipos de relaciones

Relación varios-a-uno



¿de qué tipo es cada relación?

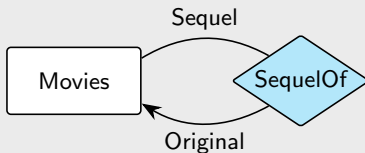


Modelo E/R: roles

Roles en una relación



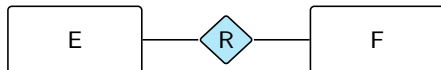
Ejemplo de roles



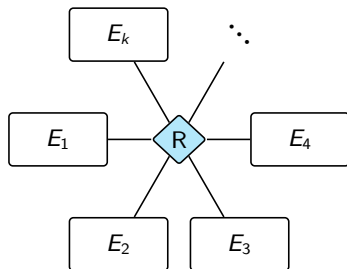
Una misma entidad puede contribuir con **múltiples roles** en una **relación**.

Modelo E/R: relaciones k -arias

Relaciones binaria

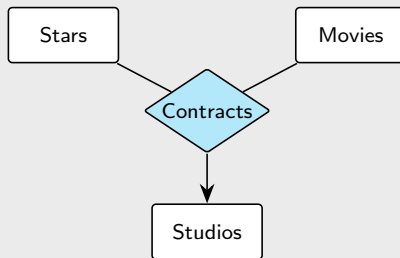


Relaciones k -aria



Modelo E/R: relaciones k -arias

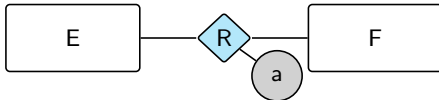
Ejemplo de una relación ternaria



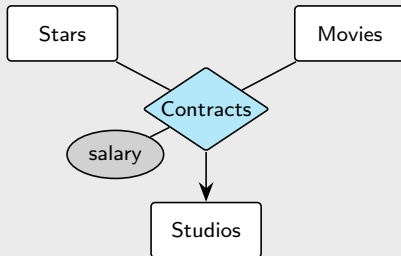
¿cuál es el significado de la **flecha** en una relación k -aria?

Modelo E/R: atributos en relaciones

Relaciones puede tener atributos

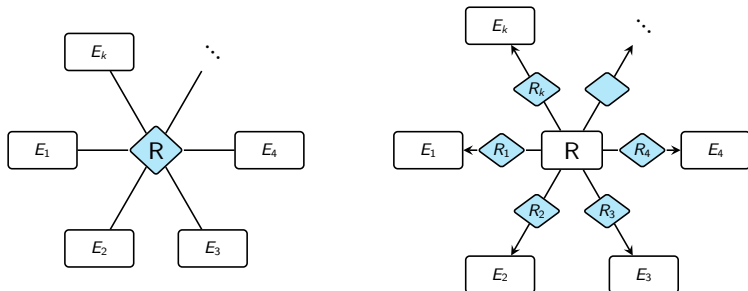


Ejemplo de una relación ternaria con atributos



Modelo E/R: relaciones k -arias versus binarias

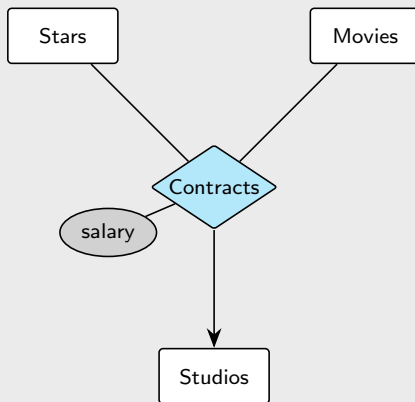
Desde relaciones k -arias a binarias



Creamos una entidad que **representa** a la relación k -aria

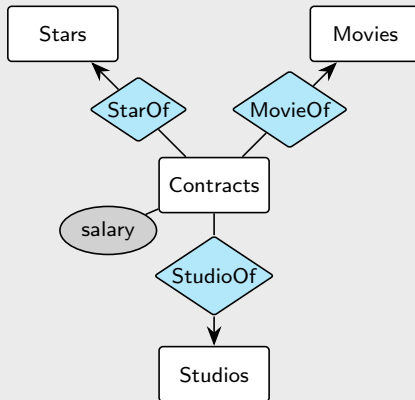
Modelo E/R: relaciones k -arias versus binarias

Ejemplo de convertir relaciones a entidades



Modelo E/R: relaciones k -arias versus binarias

Ejemplo de convertir relaciones a entidades



¿cuáles son las **ventajas** / **desventajas** de pasar a relaciones binarias?

Otros artefactos del modelo E/R

- Subclases (IsA)
- Referencias necesarias
- ...

Estos conceptos serán vistos en la ayudantía

Outline

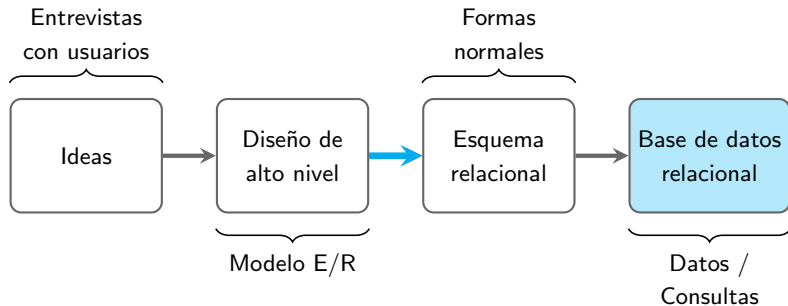
Modelo E/R

De modelo a esquema

Como llegar a un buen diseño

Llaves foráneas

Proceso de creación de una base de datos



¿cómo pasamos **del modelo E/R al esquema relacional**?

Desde el modelo E/R al esquema relacional

Estrategia directa

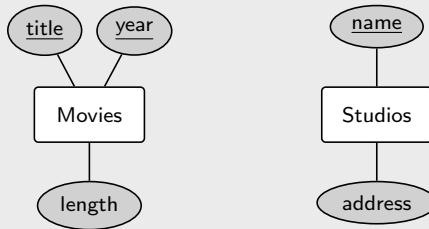
1. Convertir cada **entidad y sus atributos** en una relación.
2. Convertir cada **relación** del modelo en una relación cuyos atributos son las llaves de las entidades conectadas.

La **conversión** es prácticamente automática

(con la excepción de algunos casos)

Desde el modelo E/R al esquema relacional

Ejemplo de conversión de entidades y atributos



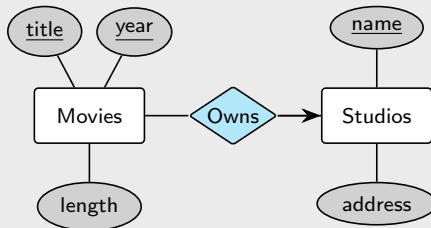
```
CREATE TABLE Movies ( title CHAR(50), year INT, length INT,  
                      PRIMARY KEY (title, year) );
```

```
CREATE TABLE Studios ( name CHAR(50), address CHAR(200),  
                        PRIMARY KEY (name) );
```

Las llaves de las entidades definen las PRIMARY KEYs de las relaciones

Desde el modelo E/R al esquema relacional

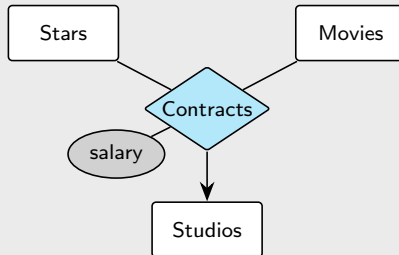
Ejemplo de conversión de relaciones del modelo



```
CREATE TABLE Owns ( titleMovies CHAR(50), yearMovies INT,  
                    nameStudios CHAR(50),  
                    PRIMARY KEY ( titleMovies, yearMovies,  
                                   nameStudios ) );
```

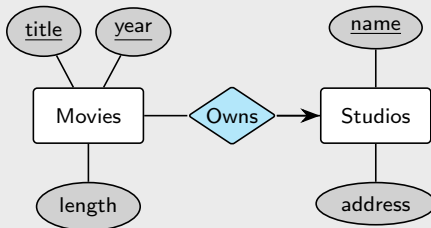
Desde el modelo E/R al esquema relacional

Ejemplo de conversión de relaciones k -arias

[illegible]

Conversión para relaciones varios-a-uno

Ejemplo de conversión de relaciones varios-a-uno

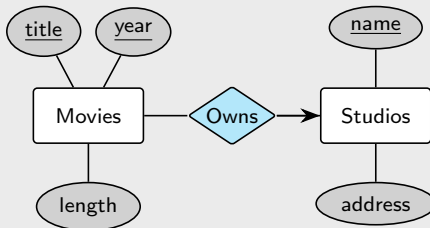


```
CREATE TABLE Owns ( titleMovies CHAR(50), yearMovies INT,  
                    nameStudios CHAR(50),  
                    PRIMARY KEY ( titleMovies, yearMovies,  
                                nameStudios ) );
```

¿es necesario crear una **relación nueva** Owns?

Conversión para relaciones varios-a-uno

Ejemplo de conversión de relaciones varios-a-uno

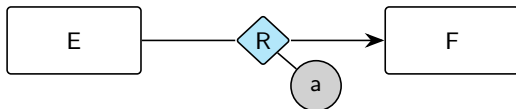


```
CREATE TABLE Movies ( title CHAR(50), year INT, length INT,  
                      nameStudios CHAR(50),  
                      PRIMARY KEY (title, year) );
```

Una relación varios-a-uno puede ser visto como una **dependencia funcional**

Conversión para relaciones varios-a-uno

Conversión de relaciones varios-a-uno / uno-a-uno



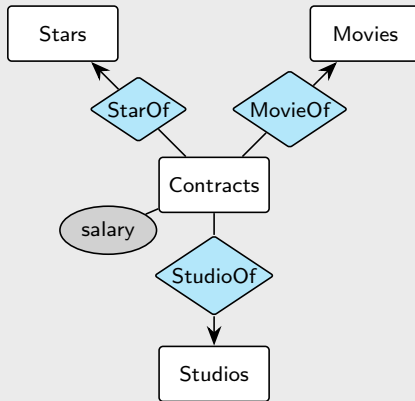
La **relación** E se crea con:

- todos los atributos de E
- + las llaves de F
- + los atributos de R .

En general, esta **solución es recomendada** para relaciones varios-a-uno

Entidades débiles

¿cómo convertimos Contracts a una relación?

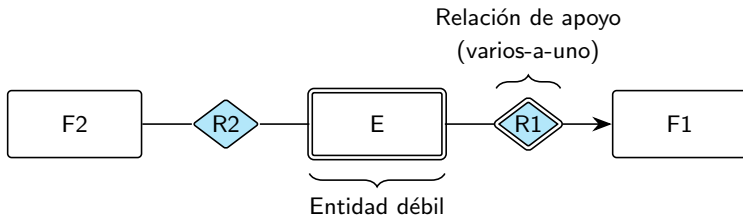


¿puede una entidad **no tener una llave**?

Entidades débiles

Definición

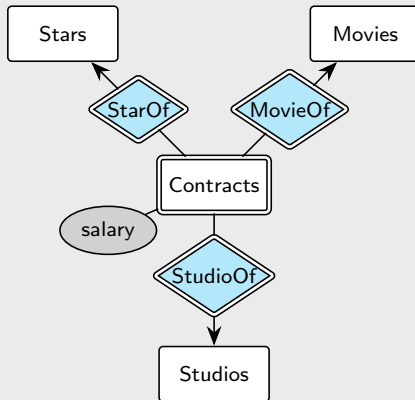
Una **entidad débil** es una entidad que **sus atributos no forman una llave**, dependiendo de otras relaciones para formar una llave.



Relaciones de apoyo permiten **completar** la llave de una entidad débil

Entidades débiles

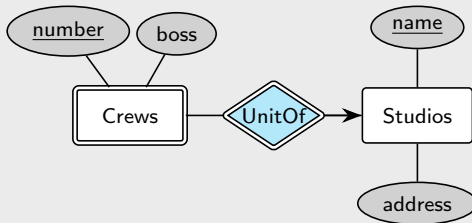
Ejemplo de entidad débil (binarización)



Ejercicio: Muestre como convertir Contracts a un esquema relacional

Entidades débiles

Ejemplo de entidad débil (caso práctico)



"Cada escuadrilla tiene un número único según el estudio al que pertenece."

Por lo tanto, dos escuadrillas de distintos estudios podrían tener el mismo número.

Outline

Modelo E/R

De modelo a esquema

Como llegar a un buen diseño

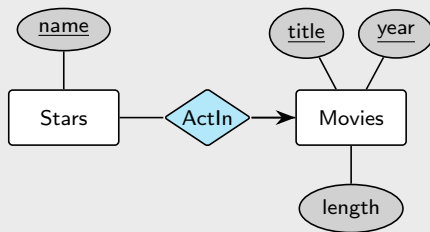
Llaves foráneas

Recomendaciones para llegar a un buen diseño

1. Fidelidad al dominio

“Las entidades, atributos y relaciones deben reflejar la realidad.”

¿cuál es el problema con este diseño?

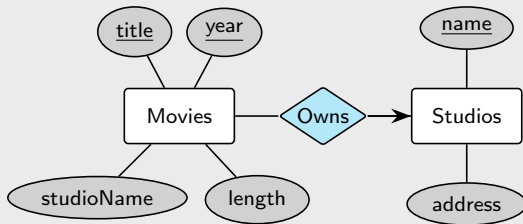


Recomendaciones para llegar a un buen diseño

1. Fidelidad al dominio
2. Evitar la redundancia

“Debemos tener cuidado de decir cada cosa una sola vez.”

¿cuál es el problema con este diseño?

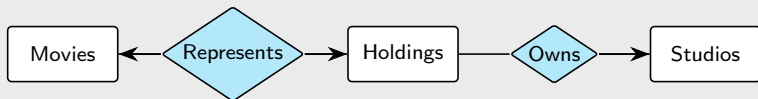


Recomendaciones para llegar a un buen diseño

1. Fidelidad al dominio
2. Evitar la redundancia
3. La simplicidad cuenta

"Evita introducir más elementos en el diseño de los estrictamente necesarios."

¿cuál es el problema con este diseño?



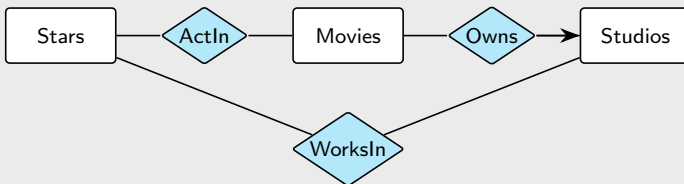
"Holdings son los representantes únicos de una película."

Recomendaciones para llegar a un buen diseño

1. Fidelidad al dominio
2. Evitar la redundancia
3. La simplicidad cuenta
4. Elegir las relaciones adecuadas

“Añadir todas las relaciones posibles no suele ser una buena idea.”

¿cuál es el problema con este diseño?

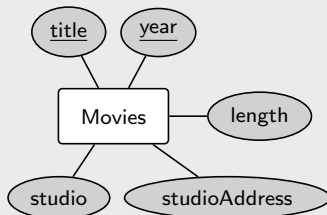


Recomendaciones para llegar a un buen diseño

1. Fidelidad al dominio
2. Evitar la redundancia
3. La simplicidad cuenta
4. Elegir las relaciones adecuadas
5. Elegir el tipo de elemento adecuado

“Escoger correctamente entre entidades y atributos.”

¿cuál es el problema con este diseño?

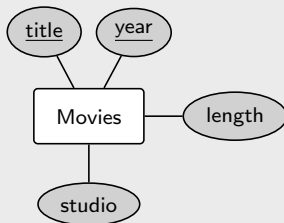


Recomendaciones para llegar a un buen diseño

1. Fidelidad al dominio
2. Evitar la redundancia
3. La simplicidad cuenta
4. Elegir las relaciones adecuadas
5. Elegir el tipo de elemento adecuado

“Escoger correctamente entre entidades y atributos.”

¿y con este diseño?



Recomendaciones para llegar a un buen diseño

1. Fidelidad al dominio
2. Evitar la redundancia
3. La simplicidad cuenta
4. Elegir las relaciones adecuadas
5. Elegir el tipo de elemento adecuado

¡Escoger un buen diseño **es un arte!**

(son solo recomendaciones y
y su aplicación depende del problema)

Outline

Modelo E/R

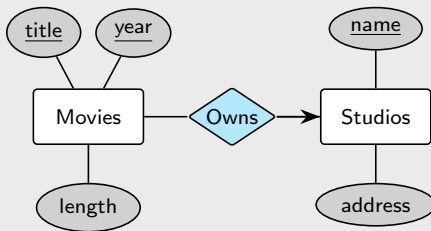
De modelo a esquema

Como llegar a un buen diseño

Llaves foráneas

Necesidad de llaves foráneas

Ejemplo de conversión de relaciones varios-a-uno



```
CREATE TABLE Movies ( title CHAR(50), year INT, length INT,  
                      nameStudios CHAR(50),  
                      PRIMARY KEY (title, year) );
```

¿cómo nos aseguramos que **Movies apunta** a **Studios válidos**?

Dependencias de inclusión

Sea $\mathbb{S} = (\mathbf{N}, \text{arity})$ un esquema relacional.

Definición

Una **dependencia de inclusión** sobre $\mathbb{S}$ es una **regla**:

$$R[A_1 A_2 \dots A_n] \subseteq S[B_1 B_2 \dots B_m] \quad \text{o} \quad R[\bar{A}] \subseteq S[\bar{B}]$$

tal que $R, S \in \mathbf{N}$, $A_1, \dots, A_n \in \text{ar}(R)$ y $B_1, \dots, B_m \in \text{arity}(S)$.

Ejemplos de dependencias de inclusión

Movies[nameStudios] $\subseteq$ **Studios**[name]

Movies[title, year] $\subseteq$ **Contracts**[titleMovies, yearMovies]

Movies[length] $\subseteq$ **Contracts**[salary]

Dependencias de inclusión

Sea $\mathbb{S} = (\mathbf{N}, \text{arity})$ un esquema relacional.

Definición

Una **dependencia de inclusión** sobre $\mathbb{S}$ es una **regla**:

$$R[A_1 A_2 \dots A_n] \subseteq S[B_1 B_2 \dots B_m] \quad \text{o} \quad R[\bar{A}] \subseteq S[\bar{B}]$$

tal que $R, S \in \mathbf{N}$, $A_1, \dots, A_n \in \text{ar}(R)$ y $B_1, \dots, B_m \in \text{arity}(S)$.

Una base de datos D **satisface** $R[\bar{A}] \subseteq S[\bar{B}]$, esto es, $D \models R[\bar{A}] \subseteq S[\bar{B}]$ si

$$\pi_{\bar{A}}(\mathcal{R}) \subseteq \pi_{\bar{B}}(\mathcal{S})$$

donde $\mathcal{R} = D(R)$ y $\mathcal{S} = D(S)$.

Dependencias de inclusión

Definición

Una base de datos D **satisfice** $R[\bar{A}] \subseteq S[\bar{B}]$, esto es, $D \models R[\bar{A}] \subseteq S[\bar{B}]$ si

$$\pi_{\bar{A}}(\mathcal{R}) \subseteq \pi_{\bar{B}}(\mathcal{S})$$

donde $\mathcal{R} = D(R)$ y $\mathcal{S} = D(S)$.

Ejemplo de dependencia de inclusión

$$D \not\models \text{Movies}[\text{nameStudios}] \subseteq \text{Studios}[\text{name}]$$

Movies

title	year	length	nameStudios
La Odisea	2026	172	Syncopy
La Odisea	1997	176	Hallmark
Oppenheimer	2023	180	Syncopy
Sharknado	2013	86	Asylum
Pecadores	2025	137	Proximity

Studios

name	address
Syncopy	4000 Warner Blvd
Hallmark	3300 W. Olive Avenue
Asylum	170 S Flower St



Dependencias de inclusión

Definición

Una base de datos D **satisfice** $R[\bar{A}] \subseteq S[\bar{B}]$, esto es, $D \models R[\bar{A}] \subseteq S[\bar{B}]$ si

$$\pi_{\bar{A}}(\mathcal{R}) \subseteq \pi_{\bar{B}}(\mathcal{S})$$

donde $\mathcal{R} = D(R)$ y $\mathcal{S} = D(S)$.

Ejemplo de dependencia de inclusión (2)

$$D \models \text{Movies}[\text{nameStudios}] \subseteq \text{Studios}[\text{name}]$$

Movies

title	year	length	nameStudios
La Odisea	2026	172	Syncopy
La Odisea	1997	176	Hallmark
Oppenheimer	2023	180	Syncopy
Sharknado	2013	86	Asylum
Pecadores	2025	137	Proximity

Studios

name	address
Syncopy	4000 Warner Blvd
Hallmark	3300 W. Olive Avenue
Asylum	170 S Flower St
Proximity	3495 Buckhead Loop



Llaves foráneas en SQL

```
CREATE TABLE <nombre-rel> (  
    <nombre-att1> <tipo1>, ..., <nombre-attN> <tipoN>,  
    PRIMARY KEY (<nombre-llave1>, ..., <nombre-llaveM>),  
    FOREIGN KEY (<att1>, ..., <attK>)  
        REFERENCES <rel>(<att1'>, ..., <attK'>) );
```

Comentarios

- <att1'>, ..., <attK'> tiene que ser una **PRIMARY KEY** en <rel>
- FOREIGN KEY define una **dependencia de inclusión**:

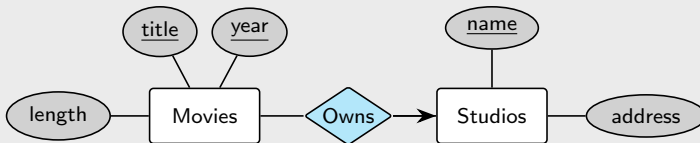
$$\langle \text{nombre-rel} \rangle [\langle \text{att1} \rangle, \dots, \langle \text{attK} \rangle] \subseteq \langle \text{rel} \rangle [\langle \text{att1}' \rangle, \dots, \langle \text{attK}' \rangle]$$

- Es posible **agregar varias llaves foráneas**.

Llaves foráneas en SQL

```
CREATE TABLE <nombre-rel> (  
    ...  
    FOREIGN KEY (<att1>, ..., <attK>  
        REFERENCES <rel>(<att1'>, ..., <attK'>) );
```

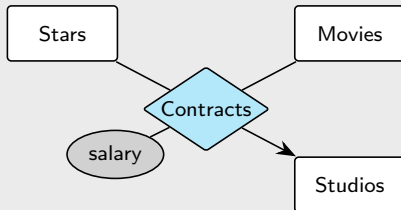
Ejemplo de uso de llaves foráneas



```
CREATE TABLE Movies ( title CHAR(50), year INT, length INT,  
    nameStudios CHAR(50),  
    PRIMARY KEY (title, year),  
    FOREIGN KEY (nameStudios)  
        REFERENCE Studios(name) );
```

Llaves foráneas en SQL

Ejemplo de conversión de relación con llaves foráneas



```
CREATE TABLE Contracts (  
    titleMovies CHAR(50), yearMovies INT, salary INT,  
    nameStudios CHAR(50), nameStars CHAR(50),  
    PRIMARY KEY(titleMovies, yearMovies,  
                nameStudios, nameStars),  
    FOREIGN KEY (titleMovies, yearMovies)  
        REFERENCE Movies(title, year),  
    FOREIGN KEY (nameStudios) REFERENCE Studios(name),  
    FOREIGN KEY (nameStars) REFERENCE Stars(name) );
```

Llaves foráneas en SQL: inserción / eliminación

```
CREATE TABLE <nombre-rel> (  
    ...  
    FOREIGN KEY (<att1>, ..., <attK>)  
        REFERENCES <rel>(<att1'>, ..., <attK'>) );
```

¿qué ocurre al **insertar / eliminar tuplas** en la tabla <rel>?

Llaves foráneas en SQL: inserción / eliminación

```
CREATE TABLE <nombre-rel> (  
    ...  
    FOREIGN KEY (<att1>, ..., <attK>)  
        REFERENCES <rel>(<att1'>, ..., <attK'>)  
        ON DELETE <comando> );
```

Reglas para insertar / eliminar tuplas

INSERT: Al insertar en <nombre-rel>, los valores <att1>, ..., <attK> deben existir en <rel>.

- Si no, ERROR.

DELETE: Al eliminar en <rel> hay que actualizar tuplas en <nombre-rel>.

- ON DELETE NO ACTION: ERROR si hay tuplas en <nombre-rel>.

ON DELETE NO ACTION es el **default** (en general)

Llaves foráneas en SQL: inserción / eliminación

```
CREATE TABLE <nombre-rel> (  
    ...  
    FOREIGN KEY (<att1>, ..., <attK>)  
        REFERENCES <rel>(<att1'>, ..., <attK'>)  
        ON DELETE <comando> );
```

Reglas para insertar / eliminar tuplas

INSERT: Al insertar en <nombre-rel>,
los valores <att1>, ..., <attK> deben existir en <rel>.

- Si no, ERROR.

DELETE: Al eliminar en <rel> hay que actualizar tuplas en <nombre-rel>.

- ON DELETE **NO ACTION**: ERROR si hay tuplas en <nombre-rel>.
- ON DELETE **SET NULL**: convierte valores en <nombre-rel> a NULL.
- ON DELETE **CASCADE**: elimina las tuplas en <nombre-rel>.